**CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)**
**FACULTY OF TECHNOLOGY AND ENGINEERING**
**SUBJECT: WEB DEVELOPMENT FRAMEWORKS (ITUE203)**
**SEMESTER: 3RD, 2026-27 (ODD)**

# Practical - 13

## PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

**Problem Definition:** Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

## A. Theory

**1. What is a JSON API?**

Answer: A JSON API allows a frontend and backend to exchange structured data in JSON format.

**2. What is Fetch API?**

Answer: Fetch is a JavaScript interface for making asynchronous HTTP requests and handling responses using promises.

**3. What does CRUD without page reload mean?**

Answer: JavaScript sends a request to PHP, receives JSON, and updates only the relevant part of the page instead of reloading the whole document.

**4. Why use HTTP status codes and error handling?**

Answer: Status codes communicate outcomes consistently, while structured error handling helps the frontend display useful messages and diagnose failures.

## B. Implementation / Procedure

**1. API endpoint**

Create a PHP endpoint that sets Content-Type: application/json, checks the HTTP method/action, and returns JSON for every outcome.

**2. GET list**

Connect to MySQL using MySQLi and return a list of students or events as JSON.

**3. Create/update/delete**

Read JSON request bodies using json_decode(file_get_contents('php://input'), true) when appropriate. Validate values and use prepared INSERT, UPDATE, and DELETE statements.

**4. Frontend fetch**

Use fetch() with the appropriate method, headers, and JSON.stringify() body. Check response.ok and parse response.json().

**5. Update interface**

Render returned records into the page and show loading, success, empty, and error states. Avoid inserting untrusted data with unsafe innerHTML.

**6. Test API**

Use browser DevTools Network tab to verify request method, payload, status code, response JSON, and that the page did not reload.

## C. Test Cases

| Test Case | Input / Action | Expected Result |
|---|---|---|
| List records | Open page | GET returns valid JSON and list renders |
| Add record | Submit valid data | POST succeeds and list updates without reload |
| Invalid data | Submit missing/invalid fields | API returns an error and no row is inserted |
| Update record | Edit a record | Database and UI show updated data |
| Delete record | Delete a record | Record disappears without full-page reload |

## D. Key Questions / Analysis / Interpretation

**1. Do PHP endpoints return valid JSON responses?**

Answer: The endpoint sets an application/json content type and returns a consistent JSON object, including success status, message, and data where applicable.

**2. Are GET and POST requests handled properly?**

Answer: GET is used to retrieve data; POST can create records. Update and delete may use POST with an explicit action or appropriate HTTP methods, with server-side validation.

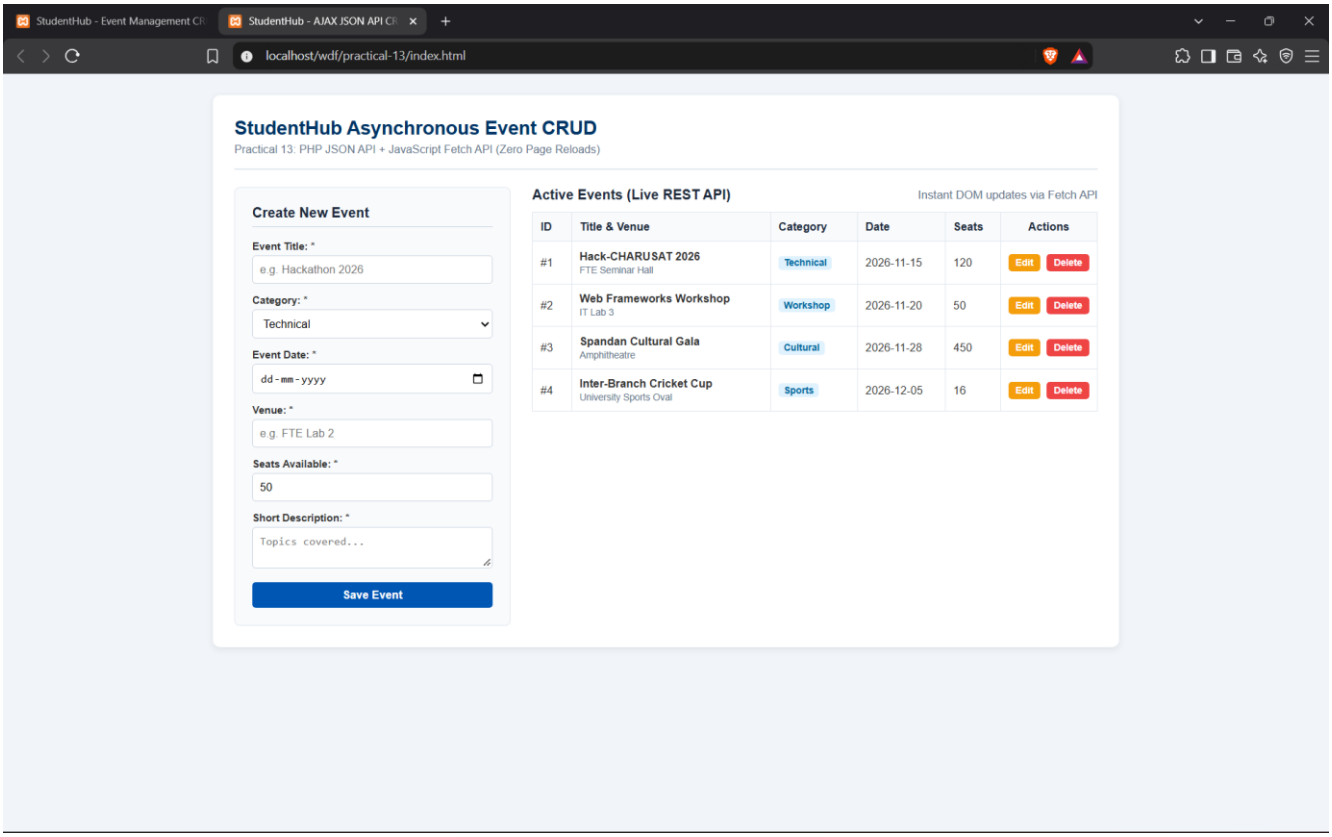
**3. Are loading, success, and error states shown on the frontend?**

Answer: The interface shows loading feedback during a request, confirms successful operations, and displays an error if the request or server fails.

**4. Is input validation performed before database operations?**

Answer: The PHP endpoint validates every field and identifier before running a prepared database query. Frontend validation is not sufficient on its own.

## M. Evidence of Work (Screenshots)




## GitHub Repository Link

https://github.com/dhyandesai1727/25DIT012-WDF-BATCH-A